- $c, \log n, n, n\log n, n^2, n^3 \ldots 2^n$
- O, Ω, Θ, o
- best-case, worst-case, average-case

! List, Stack, Queue → Dilden bağımsız yapılar.
ADTs (Abstract Data Types)

C, C++, Python, JS
C#
Java
TS

collection → STL (Standart Template Library)

List → ADT'da listeyi nasıl oluşturduğunu söylemez.
↳ create (bir liste oluştur)
insert
delete
update

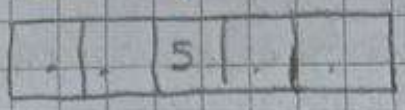
Stack (LIFO) → Stack'e ihtiyaç varsa en son işlenen elemana yeniden ihtiyaç duyulacak demektir. pop ve push işlemi var. Son giren ilk çıkar.

! set yapısı bir ağaç yapısı. Tam bir bst değil.
! vector yapısı, çalışma zamanında ihtiyaca göre otomatik büyüyüp küçülebilen dizi.
! Map (Mapping) → Hash
! Deque (Çift uçlu kuyruk) → Elemanı ekleme ve çıkarma işlemlerinin hem baştan hem de sondan yapılabildiği bir veri yapısıdır.
! Queue → FIFO → İlk giren ilk çıkar.

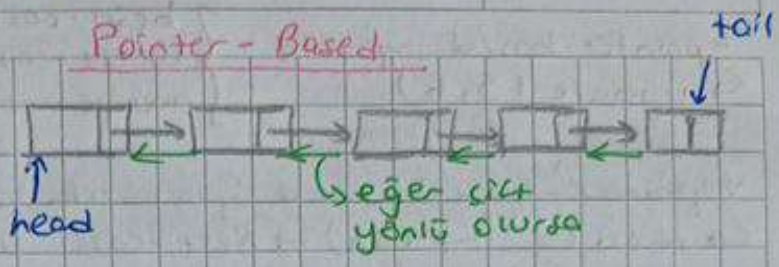
* C#'ta bir integer array oluştururken başlangıç değeri vermezsek otomatik (defaut değerler) 0 veriyor. int sayılar = new int[5]'ş birimlik yer ayırıyor.
* C#'ta silme işlemi yok. Default değer veriyor.
(ts.js'de splice ile yapılıyor.)

Array

size, type



Pointer - Based



Array

insert(5) → En sona ekleniyor.
insert(2,5) → 2. indisteki yere 5'i ekle.
} Insert örneği bu yapıya bir şey ekleme.

↳ Array cinsi olduğunda karmaşıklık:
O(1) → çünkü indisi verip bellekte sıralı olan yere ekleriz.

↳ Array class ise en son eklenecek eleman aynı. Yani insert(5) ve insert(2,5) farklı değil.
O(1) O(1)

* arrayda delete → delete(5)
↳ Arraylerde böyle bir özellik olmadığı için tanımsız (maliyet)

* Arrayde update
↳ update(2,6) → maliyeti O(1) → 2. indekse 6 elemanını ekle.

* Arrayde read
↳ read(2) → maliyet O(1)
↳ index

Pointer - Based

+ insert(5) → Başa, sona veya pointer yapısı sıralıysa sırasına uygun bir yere eklenebilir.
↳ insert(5) başa eklenirse O(1)'dir. Sona eklersek O(n)'dir. Eğer 5'i sıralı olan uygun yere koyarsak ortalamasını söyleyebiliriz. Maliyet O(n)'dir n/2 demiştik ama O(n) olarak yazıyoruz.
ortalama

* insert(5)'te sona eklerken maliyeti O(n) değil de O(1) olsun istersek çift yönlü bir liste kullanmamız gerekir. Sadece head eklersek 1 pointer, head ve tail eklersek 2 pointer olur. Çift yönlü yapı olursa 4 pointer gerekir.

* Pointer-based'de delete işlemi:
Best-case → O(1)
Worst-case → O(n)
Average-case → O(n)
↳ n/2
} Bütün yapıyı ele alıyorsak O(n)'e gider.

! comparison of list data structures

! Hepsi dizide yazıyorsak array'de delete kısmı sıkıntılı. Ona göre görülebilir.

* pointer-based update
↳ update (5, 6) → 5'i arıcak

best-case : O(1)
worst-case : O(n)
average-case : O(n) → n/2

* pointer-based read (pointerda indeks yok)
↳ read (2) → O(n) → Best-case O(1) ama gerçekte söylemez.
2 olan elemanı getirir. (pointer yapısı)

! linked list (pointer yapısı) ve array ile oluşunca maliyet değişiyor.

List → array, pointer
insert
delete → array'de delete yok.
update
read

! Array'de delete'i temiz hale getirmek için:

delete: shift → O(n) → ts'deki since da aynı şey yapıyor.
derivative → O(1), memory problemi

↳ shift → 1 2 ~~5~~ 4 3 6 → O(n) = worst-case
5'i sil 5'ten sonrakileri bütün olarak kaydır.
O(1) best-case'i ayrıca raporlamaya gerek yok.

↳ derivative → 1, 2, -1, 4, 3 → O(1) olsa bile memory problemi var.
↳ silinen elemanı yerine -1 kondu.

Stack

	Array	Pointer-Based
#read pop() →	O(1)	O(n), O(1)
		sadece head varsa / head ve tail (baş sonu gösteren 2 pt.) varsa
#write push(5) →	O(1)	O(n), O(1)

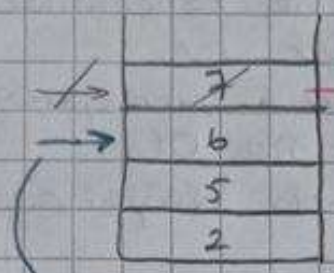
read'de değişiyor.

pop(): sondaki elemanı al gibi. read ile benzer

push(5): write ile benzer. insert'e de benziyor.

! array veya pointer olmasına göre → insert, delete, update, read işlemleri değişiyor.

* pop (stack) → en sondan siliyor.



→ 7'yi alıyor. Read işlemiyle delete işlemi aynı anlam.

→ en sondaki elemanı gösteren yapıyı 1 aşağı getir.

! Stack'te pointer-based push:
O(n) → en sona ekleyeceksek
O(1) → eğer en sonu da gösteren bir pointer (ta) varsa O(1)

Queue

	Array	Pointer-based
#write (dizinin en sonuna eleman ekler) enqueue →	O(1)	O(n), O(1)
#read (dizinin başına eleman ekler) dequeue →	? (burun yerine shift veya derivative)	direkt O(1) diyebiliriz. O(1), O(1) #head güncellendi. (tail varsa)

↳ kuyruğun başını okuyacak.

array based'de kuyruk yapı yok

↳ veri okunup çıkarılınca 0. indexte başı oluyor.

! Arrayin başını sil diye bir yapı yok. Onun için yine shift veya (bu yapılar var) derivative gibi şeyler kullanılmalı.

! liste, kuyruk, stack yapıları var. Listeye wrapper yazarak stack ve kuyruğa gidiliyor.

↳ wrapper (sarmalayıcı): Var olan bir veri yapısını (örn. liste) kullanıp onun etrafına yeni bir yapı gibi davranan bir sınıf / arayüz sarmalıdır.

CRUD

List → insert
delete
update / read

Queue → .read → dequeue
.write → enqueue

Stack → .read → pop
.write → push(...)

Stl ve collectionlarda bunlar var.
Bunların maliyetleri O(1) ile O(n) arasında değişiyor.